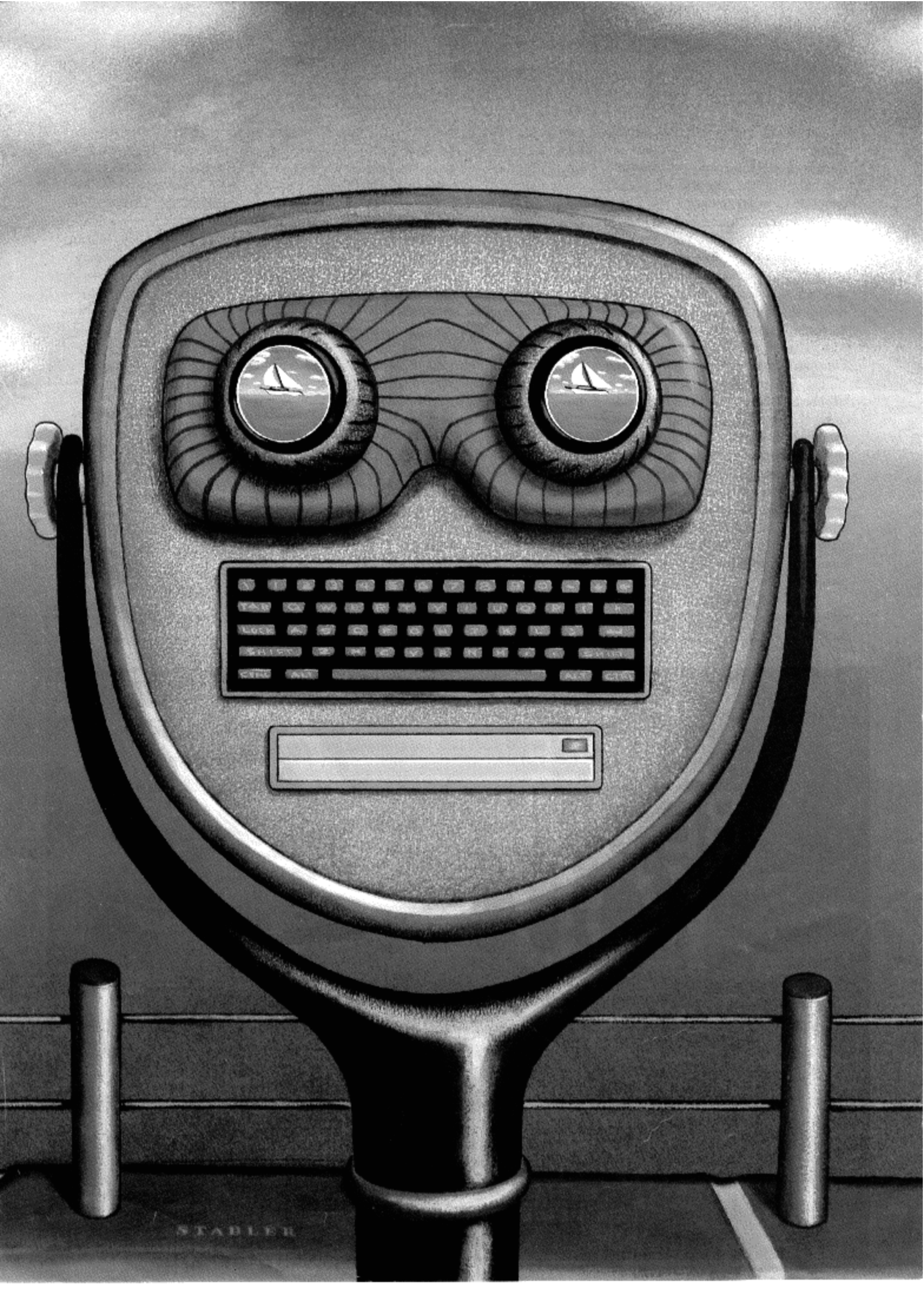




STABLER



Making *Use* A Design Representation

Computers are more than just narrow functionality. They unavoidably restructure human activities, creating new possibilities as well as new difficulties. Conversely, each context in which humans experience and act provides detailed constraint for the development and application of computer technologies. In analyzing and designing systems and software, we need better means of talking about how they may transform and/or be constrained by the contexts of user activity—this is the only way we can hope to attain control over the design of useful and usable systems. A direct approach is to explicitly envision and document typical and significant user activities early and continuingly in the development process. Such descriptions, often called “scenarios,” support reasoning about situations of use, even before those situations are actually created.

In this article, I survey and illustrate a method in which scenarios provide an integration of task analysis and design envisionment—a basic level of design representation. Scenario representations can be elaborated as prototypes, through the use of storyboard, video, and rapid prototyping tools. They are the minimal contexts for developing user-oriented design rationale: a given design decision can be evaluated and documented in terms of its specific consequences within particular scenarios. Scenarios and the elements of scenario-based design rationale can be generalized and abstracted using theories of human activity, enabling the accumulation and development of knowledge attained in the course of design. Thus, scenarios can provide a framework for a design-based science of human-computer interaction (see [1]).

Scenario-Based Design: A Computer Is What One Does With It

To understand the consequences of computer technology for people, it is necessary to see computers as artifacts of human activity, as educational environments, as work tools, as media for communication. We can do this empirically; that is, we can learn from our experiences in deploying technology just what works and what does not work. We can also do this analytically: We can collect and consider user requirements, evaluate representations and prototypes, and build design rationales. User scenarios can be a key to all such

analyses: Traditionally, we tried to see computers as artifacts in human activity through functional specifications, but these provide a highly idealized view of use (e.g., features organized alphabetically, function by function)—not a view of how a user would engage the function and experience its resulting effects.

The real use of systems can be concretely described by the set of user-interaction scenarios obtained when people try to use a system. Box 1 presents an observed example of an interaction scenario. An experienced procedural programmer is using a Smalltalk/V PM tutorial, working on an exercise to implement a new kind of text pane that displays its input in uppercase. This project focuses on concrete, albeit somewhat arcane, functionality; it involves learning about two key subtrees of the Smalltalk class hierarchy (Magnitude-Character and Window-SubPane-TextPane) and allows the programmer to implement a concrete and usable new Smalltalk object by extending existing system functionality

with a minimum of coding. The programmer carries out the project by subclassing the TextPane class and specializing the code that processes character input from the keyboard. The scenario is a vivid and succinct description of a person learning something about Smalltalk; a comprehensive set of such scenarios would be a very concrete task analysis of the Smalltalk tutorial situation.

Even this single scenario is an obvious and immediate design resource: It highlights goals suggested by the appearance and behavior of the system, what people try to do with the system, what procedures are adopted, not adopted, carried out successfully or erroneously, and what interpretations people make of what happens to them. One can use it to identify problem areas for further design work. For example, in Box 1, the programmer has some difficulty navigating the Smalltalk class hierarchy—the tutorial could be improved by incorporating an indexing tool. A more subtle example, but an important one in the Smalltalk do-

main, is that the programmer in Box 1 copied a method to a subclass instead of inheriting the functionality and merely implementing the specialization in the subclass.

Projecting scenarios in design. We cannot rely only on observation to build a scenario representation. If we want scenarios to play a proactive role in guiding design and development, we need to be able to construct scenario representations for systems *before* any version is deployed in the field [7, 16]. In this case we are representing anticipated use. We can do this in part by analogy: The new system may be similar to other contemporary systems or “descended” from earlier systems—the characteristic scenarios for these related existing systems *can* be observed and analyzed.

We can also make use of theories of human activity [17]; that is, we can try to instantiate known categories of human activity in the envisioned context of the system we are developing. For example, people must *orient* themselves to novel task situations, identifying and analyzing appropriate goals; later, they must *evaluate* their own actions to determine whether those goals were attained (e.g., [9, 17]). In building a scenario representation for a system in development, we can envision possible goal orientation and evaluation situations. As we develop such scenarios, we can track specific impacts of our design decisions for these theoretical categories of human activity.

The novelty of this proposal resides simply in the suggestion that scenarios be systematized as working design representations, for it is well known that scenarios are spontaneously and pervasively used in system development as currently practiced. Studies of actual design situations document the use of fairly detailed conjectures about what users will do with the system [11, 15, 21].

It has also been shown that users and designers can communicate effectively about system requirements and design options through narrative descriptions of envisioned use [12, 14]. Indeed, the current state of practice more or less requires a substantial set of scenarios both for usability test-

Box 1. A Smalltalk tutorial scenario “Creating UpperCasePane”:

A set of such scenarios—those suggested by the tutorial manual or system software and those observed or anticipated—comprises a use-oriented representation of the Smalltalk tutorial.

A beginning Smalltalk programmer is working through a series of tutorial projects designed to provide initial familiarity with some central classes and methods. The programmer is trying to create a new kind of textpane that displays keyboard input in uppercase.

The programmer searches for existing relevant functionality, simultaneously trying to define an approach to the project and to identify existing software resources for it. Eventually, the programmer locates the methods `asUpperCase` (in the `Character` class) and `characterInput:` (in the `TextPane` class). The sheer number of possibly relevant classes and methods necessitates considerable browsing in order to do this, and the programmer pursues several incorrect possibilities along the way (the programmer has trouble finding the `Character` class, later losing track of where it is while browsing in `TextPane`, and at first considers specializing the `appendText:` method of `TextPane`, which does not take a character argument).

The programmer specializes `TextPane` by creating a new subclass `UpperCasePane`, copying down its `characterInput:` method, and altering one line of the code to transform the `characterInput:` argument `asUpperCase`.

The programmer creates and tests an `UpperCasePane`. The programmer is satisfied with the result and decides to just move on to the next tutorial project.

ing [19] and for documentation and training design [5]. User-interaction scenarios are typical in contemporary functional specification documents (*officially* they are included as illustrations of the specification, but they often appear *before* any of the specification, and developers suggest they are sometimes used in lieu of the specification). In sum, scenarios are already prevalent in the design process. Systematically developing and managing a set of scenarios as part of the working design representation can help integrate and leverage these diverse, informal, and currently uncoordinated uses. Indeed, the costs of doing more comprehensive scenario development might actually be *less* than the distributed costs of current practice.

Our experience. We have used scenario-based design representations in a variety of recent projects designing tools and environments for Smalltalk instruction and programming [8–10, 20, 23, 24]. Our experience is therefore biased toward open-ended construction domains, although one might expect scenario-based methods would apply at least as well in more circumscribed application domains (in which the user tasks are often literally enumerable). We have found that using fewer than a dozen scenarios provides a useful and manageable design representation.

Box 2 describes one of the scenarios envisioned to drive the design of the Smalltalk Guru tool in our MoleHill intelligent tutoring system [1, 23, 24]. The Guru tool provides guidance modeled on the inclination people typically have toward evaluating their own activity: *just* as the programmer finishes the project, the Guru offers suggestions about how to achieve a more modular code design—in this scenario exploiting Smalltalk's inheritance mechanism via the special variable "super." This example shows how empirical analysis of interaction scenarios for existing situations (for example, the Smalltalk tutorial situation illustrated in Box 1) and abstractions from theories of human activity (the notion of an evaluation scenario) can provide proactive guidance in developing a scenario representation for a new

Box 2. A Smalltalk Guru scenario, contrasting "copy down" and "reference to super":

Scenarios can be written down, developed, and reasoned about before any system supporting them has been designed or implemented. The Guru "super" scenario is part of the design representation for the MoleHill intelligent tutor for Smalltalk [1, 23, 24].

As the programmer completes a tutorial project, he or she is prompted as to the availability of a Smalltalk Guru postmortem for this project.

The programmer selects the postmortem, and the Smalltalk Guru appears and explains that while the solution of copying down the superclass's characterinput: method is correct, there is another approach in which the method code is maintained only with the superclass (in this case, TextPane). The Guru demonstrates how subclasses can specialize this functionality with a message send to the special variable "super," directing the Smalltalk interpreter to begin searching for the method code in the superclass. (The programmer whispers, "So *that's* how 'super' gets used!")

At the end of the project-specific postmortem, the programmer is offered more general Guru assistance (for example, a discussion of inheritance), but decides to go on to the next project.

system.

In our work, scenarios are to some degree continually developed and reinterpreted. The scenario in Box 2 is a development of the scenario in Box 1, but it is not a final representation. It is merely a use-oriented view of the current design, a working representation. Different scenarios can conflict and "compete" in such a working design representation, forcing out greater clarity in the developing vision of the system.

Developing scenarios through the coordinated use of direct empirical observation and abstractions from theories of human activity has the additional benefit of continually emphasizing to the designer the continuities among various design projects. System design is, of course, highly creative, but few systems are completely novel. Indeed, the history of technology is a history of cumulative development and evolution [2]. Nevertheless, it is easy in the context of design work to focus singularly on the project at hand and lose the benefit of whatever might have been done and learned previously. Our deliberate development of the Guru scenario helped us appreciate the Guru as a type of bug critic (e.g., [3]) and thus

as systematically related to other tutorial situations that had been designed and studied before.

Scenario-based design representations create many challenges of their own. It is clear to most people how to build an alphabetical list of functions. Building a set of user-interaction narratives may be more appropriate, but it is also more ill defined. Traditional functional specifications are frequently inconsistent or incomplete with respect to the user's needs and predilections (this was one of the cataclysmic discoveries of the 1980s), but because they are organized in a function-by-function manner, they are at least easy to align with the detailed design and implementation of the system. Two important current frontiers for scenario-based design methodology are (1) heuristics to help ensure broad sampling of scenarios in building a representation [9], and (2) tools for verifying and managing the alignment of scenario representations with detailed designs and implementations [4, 6, 20].

Psychological Design Rationale: A Computer Is the Consequences It Has for Users
Representing the use of a system or

application with a set of user-interaction scenarios makes that *use* explicit, and in doing so orients design and analysis toward a broader view of computers. It can help designers and analysts to focus attention on the assumptions about people and their tasks that are implicit in systems and applications. But the scenarios themselves leave implicit the underlying causal relations of use: A scenario does not isolate just what it is about people or about a design that allows that scenario to occur.

Explaining scenarios. It can be valuable to make explicit a causal account or rationale for a scenario. Indeed, envisioning a "better" scenario for users may well require making explicit just what needs to be improved in terms of specific consequences for users, how improvements might be attempted in terms of managing or altering consequences, and what other consequences might result as side effects. A direct approach to articulating why scenarios run as they do is to isolate individual causal rela-

names may help programmers quickly identify relevant functionality, the names may also mislead programmers. The Navigation Relation (c) is a hypothesis that searching the class library may cause the programmer to incidentally learn something about the functionality in the library, though the sheer size of the library may make it difficult to learn much, particularly if the programmer is frustrated by searching for material. Finally, the Programming Tools Relation (d) observes that the tool-rich, interpreted Smalltalk environment may support a very fluid and opportunistic style of code development; the downside is that programmers may not reflect enough on the designs they are generating.

A psychological design rationale such as this can be developed by asking questions to elaborate the events in a use scenario [9]. Thus, for the UpperCasePane scenario (Box 1), we might ask: Why did the programmer search for relevant functionality while still defining a strategy toward the project? Did anything about the situation make it especially difficult or easy to proceed in this way? What was the programmer thinking about while browsing for relevant classes and methods? Did the programmer have any particular difficulties with browsing, or benefit from the browsing activity? Why did the programmer pursue several incorrect possibilities when searching for relevant functionality? Why was `appendText` considered, and why was it finally rejected? Why did the programmer have trouble finding, and then later lose track of, the `Character` class? Why did the programmer copy down the method code for `characterInput`: instead of inheriting the functionality using "super?" How did the programmer decide to use `Character`'s `asUpperCase` method? Why does the programmer test only one `UpperCasePane` and test only the external behavior of the pane?

The answers to such questions are a set of hypotheses about the underlying causal relations in the scenario—the analyst's hypotheses about why and how an observed (or envisioned) scenario of use could occur. The objective of raising these questions and

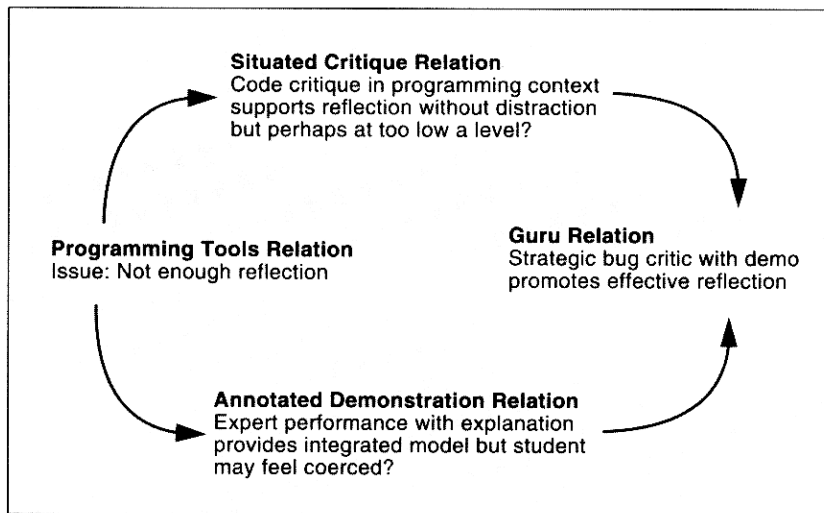


Figure 1. Design Reasoning for the Smalltalk Guru: The Programming Tools Relation highlights the possibility that programmers may not direct enough time and effort to code design. Scenario analysis of the prior system LISP-CRITIC and of the theory and technique of reflective practice brought to light a variety of issues and design ideas: providing critique within a current problem context, providing narrated demonstration of target skills to better link concepts and actions, allowing user control of critique initiation and user discretion as to consequent actions. Articulation of the causal relations in these situations guided us in combining their elements in the Smalltalk Guru, a strategic bug critic with demonstration of programming process.

tionships between features of a situation of use and specific consequences for users:

IN <situation> <a feature> CAUSES <"desirable" psychological consequence> BUT MAY ALSO CAUSE <"undesirable" psychological consequence>

Box 3 presents four causal relationships implicit in the UpperCasePane scenario (Box 1). Thus, the Code Library Relation (a) asserts that the class hierarchy encourages standard design decompositions and reuse-oriented programming strategies. These are desirable consequences for the programmer working on the tutorial project. A downside, or undesirable consequence, of the large code library is that Smalltalk programmers may experience problems searching for code to reuse or making appropriate use of code. The Names Relation (b) observes that while descriptive class and method

articulating hypothesized causal relations is to open up a process of critique and refinement, to help designers interrogate and explain key scenarios in enough detail to let them be interpreted and revised or perhaps discarded as the design proceeds. It is important to emphasize that a psychological design rationale, such as the fragment in Box 3, does not necessarily equate to the personal intentions of the designers. We know that the designers of Smalltalk actually did intend to promote code reuse [18], but it seems safe to assume that they did *not* intend to create search and reuse problems.

Using psychological design rationale. Psychological design rationale can be used as a working design representation throughout the design process. Just as in the case of scenarios, we want to use analyses of existing situations to help set the agenda for design work and to guide reasoning about particular possibilities.

The analysis in Box 3 supported our work on MoleHill by clearly articulating what it is about situations like that depicted in Box 1 that should be addressed during design. A general hillclimbing heuristic for using a psychological design rationale in this way is to envision alternate scenarios such that downside consequences of particular relations are obviated or mitigated, while upside consequences are enhanced or maintained. Thus the Guru scenario in Box 2 mitigates the downside of the Programming Tools Relation while maintaining its upside. That is, the Guru project postmortem encourages reflection on code design but does not constrain fluid and opportunistic programming activity prior to the postmortem.

Having placed on our design agenda the goal of encouraging attention to and reflection on code design without diminishing flexibility and empowerment, we explored theories and techniques pertaining to reflective activity and specific designs to promote such activity. Our object was to further elaborate the design of the Guru and its envisioned consequences for users. We studied LISP-CRITIC [13], an experimental critiquing facility to which programmers submit code fragments and receive

Box 3. Some causal relations implicit in the UpperCasePane scenario:

A causal account of specializing Smalltalk's TextPane includes possibilities for reusing code (a), opportunistic strategies (b), browsing (c), and the rich tool environment (d). (Relations (a), (b), and (c) correspond to (5j), (5k), and (5l) in [8]; relation (d) corresponds to the Task Switching Claim in [23]).

In the UpperCasePane scenario (Box 1),

(a) **Code Library Relation:** Existing code in the Smalltalk class hierarchy causes programmers to reuse standard decompositions in designing new functionality (encourages the strategy of programming by assembling parts) but may also cause search problems (because it's so big) and nonoptimal solutions (reuse of inappropriate components)

(b) **Names Relation:** Descriptive class and method names cause programmers to rely on names in searching for functionality and planning reuse but may also cause false alarms (by suggesting spurious connections)

(c) **Navigation Relation:** Navigating the class hierarchy causes unintentional learning about existing functionality but may also cause confusion and frustration (since the class hierarchy is so large)

(d) **Programming Tools Relation:** Powerful programming tools cause browsing, opportunistic problem solving, and a rapid prototyping approach to code development but may also cause programmers to direct insufficient time and effort to code design

suggestions about how to make their code easier to read and maintain or more machine efficient. The system does not place any constraints on how programmers create the code submitted for critique, nor on whether or how they use its suggestions. In our analysis, the psychological design rationale for LISP-CRITIC hinges on the possibility that a highly specific critique, delivered at the programmer's request and in the context of a current programming situation, can encourage programmers to reflect more on their work without undermining their preferred work style:

Situated Critique Relation: A situated critique on specific code design problems—at the programmer's request—causes programmers to reflect more on their code without hampering opportunistic code development but may also cause programmers to reflect on their work too narrowly (may improve a particular code fragment, but fail to teach a more general design lesson)

We had envisioned the Guru as pro-

viding a very similar sort of situated critique (though we wanted to address architectural issues, like inheritance—Box 2).

We also investigated the theory and technique of "reflective practicum" [22], in which a teacher models expert performance for a student while commenting on the process of producing such performance and coaching the student on how to move toward achieving such performance:

Annotated Demonstration Relation: The teacher's self-consciously annotated demonstration of expert performance causes the student to integrate actions and concepts in a paradigmatic model but may also cause the student to feel coerced (due to the power imbalance between teacher and student)

This kind of scenario and its characteristic relation bears on the goal of helping programmers integrate their understanding of inheritance and the use of "super" in Smalltalk with their strategies for building code. We

wished to create the upside consequences of the Annotated Demonstration Relation with the Guru and believed that we could avoid the downside consequences, since the Guru would only be a software tool initiated by the programmer.

Our experience. Design reasoning is often quite local; designers address sets of requirements and then set aside tentative reasoning and partial results in order to explore other, hopefully converging subproblems [11]. This is precisely why it may be useful to create explicit rationale: interim results and their backing can be preserved and deliberately integrated into the overall design solution. Articulating the causal relations in LISP-CRITIC and reflective practicum situations guided us in combining their elements in the Smalltalk Guru (see Figure 1). This is not to suggest that design reasoning can be reduced to a calculation; rather it is to admit that only when arguments are made explicit can they be evaluated.

In our design work on instructional and programming tools, we have explored the usefulness of articulating the causal relations of a psychological design rationale [8–10, 20, 23, 24]. We find that, like scenarios, these representations help us directly by creating auditable design claims about how an envisioned system will affect its users. A clear tradeoff is that although articulating each individual relation of a psychological design rationale creates a more precise representation than a scenario, creating such a rationale is itself a more difficult design task. We have developed a heuristic for generating elaboration questions from which to develop psychological design rationales from scenarios [9] and have found that having related rationales already in hand is a significant aid; nonetheless, developing a psychological design rationale is still a significant task. A practical compromise is to develop a full psychological design rationale only for key scenarios and cases where a scenario representation leaves open questions.

It is important to emphasize that psychological design rationales are heuristic explanations of human behavior and experience. Their utility in system design derives from directly

linking features of a designed situation with user activities and experiences. This can be an oversimplification, however: many of the causal relations between features of situations and consequences for people in those situations are not first-order relations; they derive from interactions and side effects. Our hope is that when such interactions arise, we might improve our chances of noticing them and of beginning to fathom what's going on *if* we are keeping explicit account of what we expected would happen. If something is clearly working or failing, we have a basis for making credit/blame attributions—a starting point for systematically revisiting our decisions and for understanding other situations that guided us to those decisions [10].

Making More Systematic Use of Use

In the current state of the art, managing usability and usefulness generally means performing early user testing and iterative redesign. I would take as proven that this has enhanced the system development process, and I would expect that it may always be a key element in that process. Nevertheless, it is not at all clear that we can or should seek to ensure usability and usefulness only through purely empirical means.

This article has sketched a simple but extendable analytic method for managing usability and usefulness, by making use itself a design representation. In this method, user-interaction scenarios are a concrete medium for describing existing situations and for projecting envisioned situations. A set of such scenarios is a broadly useful design representation; it is a resource for reasoning about what people will do and experience in a new situation; it is an auditable design claim about what will be delivered to the user. Such a representation can be further elaborated into a psychological design rationale, specifying causal relations among aspects of the designed situation and consequences for people in the situation.

This method attempts to leverage established practices in using scenarios to communicate with users about requirements, to illustrate functional specifications, and to manage usability

testing and documentation development. Creating scenarios more systematically and earlier in the design process as planning and reasoning aids is only an incremental elaboration of current practice. If a designer or analyst finds that making more systematic use of scenario representations is truly valuable, he or she may be motivated to consider enhancing these with some annotation of why the scenarios run as they do. If this enhancement also seems valuable, he or she may consider developing a psychological design rationale for a project. Hopefully, the gentle gradient from current (somewhat casual) use of scenarios in design practice to the somewhat more systematic use we are developing will enhance the acceptability of the method.

Finally, we have been encouraged in our view that taking a more systematic approach to the representation of use in design can support the accumulation and development of knowledge attained in the course of design. When we can view a scenario as an instance of a type, we are in a position to generalize whatever we learn about that particular scenario to the type. Thus, recognizing that the Smalltalk Guru is a kind of bug critic similar to LISP-CRITIC can serve two purposes: first, it can help us elaborate the design and the design rationale for the Guru; second, it creates the possibility that things we subsequently learn about the Guru can be mapped back to LISP-CRITIC, or more generally to bug critics, as a specific hypothesis. Similarly, drawing on general types of human activity—like orienting to goals or evaluating one's actions [17]—in generating scenarios to envision a system in design can also help us attribute things we learn about given scenarios of use for given systems to a more general analysis of human activity. Inductive generalization is the chief means for creating new knowledge, but it requires a framework for seeing a particular as an instance of a type. Some of our recent work has detailed how the framework of scenarios and psychological design rationale could leverage usability testing as the empirical backbone of a design-based science of human-computer interaction [10].

We will always need to manage the design of efficient and reliable devices. If we also take seriously seeing them as artifacts of human activity, as elements of designed situations, as participants in causal relations with human experiences, we will improve our chances for deliberately designing computers that are what we want them to be: stimulating and useful tools for people. ■

Acknowledgments

I'm grateful to Jurgen Koenemann-Belliveau, Scott Robertson, and Kevin Singley for comments on an earlier version of this article; special thanks to Mary Beth Rosson for scrutinizing two drafts.

References

1. Alpert, S.R., Singley, M.K., and Carroll, J.M. Multiple multimodal mentors: Delivering computer-based instruction via specialized anthropomorphic advisors. *Behaviour and Information Technology*. In press.
2. Basalla, G. *The Evolution of Technology*. Cambridge University Press, New York, 1984.
3. Brown, J.S., Burton, R.R., and de Kleer, J. Pedagogical, natural language and knowledge engineering techniques in SOPHIE I, II and III. In *Intelligent Tutoring Systems*, D. Sleeman and J.S. Brown Eds. Academic Press, New York, 1982, pp. 79-98.
4. Carey, T. McKerlie, D., and Wilson, J. HCI design rationales as a learning resource. In *Design Rationale*, Moran, T.P., and Carroll, J.M., Eds. Erlbaum, Hillsdale, N.J. In press.
5. Carroll, J.M. *The Nurnberg Funnel: Designing Minimalist Instruction for Practical Computer Skill*. MIT Press, Cambridge, Mass., 1990.
6. Carroll, J.M., Ed. *Scenario-based Design: Envisioning Work and Technology in System Development*. Wiley, New York, 1995.
7. Carroll, J.M., and Rosson, M.B. Human-computer interaction scenarios as a design representation. In *Proceedings of HICSS-23: Hawaii International Conference on System Sciences*. IEEE Computer Society Press, Los Alamitos, Calif., 1990, pp. 555-561.
8. Carroll, J.M., and Rosson, M.D. Deliberated evolution: Stalking the View Matcher in design space. *Human-Computer Interaction* 6 (1991), 281-318.
9. Carroll, J.M., and Rosson, M.B. Getting around the task-artifact cycle: How to make claims and design by scenario. *ACM Trans. Infor. Syst.* 10 (1992), 181-212.
10. Carroll, J.M., Singley, M.K., and Rosson, M.B. Integrating theory development with design evaluation. *Behaviour and Information Technology* 11 (1992), 247-255.
11. Carroll, J.M., Thomas, J.C., and Malhotra, A. A clinical-experimental analysis of design problem solving. *Design Studies* 1 (1979), 84-92. Reprinted in *Human Factors in Software Development*, B. Curtis, Ed. IEEE Computer Society Press, Washington, 1985, pp. 243-251.
12. Ehn, P. *Work-Oriented Design of Computer Artifacts*. Arbetslivscentrum, Stockholm, 1988.
13. Fischer, G., Lemke, A.C., Mastaglio, T., and Morch, A.I. Using critics to empower users. In *Proceedings of CHI'90 Conference* (April 1-5, Seattle, Wash.). ACM, New York, 1990, pp. 337-347.
14. Greenbaum, J., and Kyng, M., Eds. *Design at Work: Cooperative Design of Computer Systems*. Erlbaum, Hillsdale, N.J., 1991.
15. Lammers, S. *Programmers at Work*. Microsoft Press, Redmond, Wash., 1986.
16. Lewis, C.H., Rieman, J., and Bell, B. Problem-centered design for expressiveness and facility in a graphical programming system. *Human-Computer Interaction* 6 (1991), 319-355.
17. Norman, D.A. Cognitive engineering. In *User Centered System Design*. D.A. Norman and S.W. Draper, Eds. Erlbaum, Hillsdale, N.J., 1986, pp. 31-62.
18. Pope, S.T., Goldberg, A., and Deutsch, L.P. Object-oriented approaches to the software lifecycle using the Smalltalk-80 system as a CASE toolkit. In *Proceedings of the Fall Joint Computer Conference—Exploring Technology: Today and Tomorrow* (Dallas, Texas, October 25-29). IEEE Press Computer Society, 1987, pp. 13-20.
19. Roberts, T.L., and Moran, T.P. The evaluation of text editors: Methodology and empirical results. *Commun. ACM* 26 (1983), 265-283.
20. Rosson, M.B., and Carroll, J.M. Extending the task-artifact framework. In *Advances in Human-Computer Interaction*, 4, H.R. Hartson and D. Hix, Eds., Ablex, Norwood, N.J., 1993, pp. 31-57.
21. Rosson, M.B., Maass, S., and Kellogg, W.A. The designer as user: Building requirements for design tools from design practice. *Commun. ACM* 31 (1988), 1288-1298.
22. Schön, D. *Educating the Reflective Practitioner*. Jossey-Bass, San Francisco, 1987.
23. Singley, M.K., and Carroll, J.M. Synthesis by analysis: Five modes of reasoning that guide design. In *Design Rationale*, T.P. Moran, and J.M. Carroll, Eds. Erlbaum, Hillsdale, N.J. In press.
24. Singley, M.K., Carroll, J.M., and Alpert, S.R. Psychological design rationale for an intelligent tutoring system for Smalltalk. In *Empirical Studies of Programmers, Fourth Workshop*, J. Koenemann-Belliveau, T. Moher, and S.P. Robertson, Eds. Ablex, Norwood, N.J., 1991, pp. 196-209.

About the Author:

JOHN M. CARROLL is professor of computer science and psychology and head of the computer science department of Virginia Polytechnic Institute and State University, Blacksburg, Va. This research was carried out while he was manager of design concepts and education at the IBM Watson Research Center. Current research interests include human-computer interaction, design concepts and methods for usable systems and software, and the history and philosophy of technology. **Author's Present Address:** Department of Computer Science, 562 McBryde Hall, Virginia Tech (VPI&SU), Blacksburg, VA 24061-0106; email: carroll@cs.vt.edu

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

© ACM 0002-0782/94/1200 \$3.50